



المعهد الوطني للبريد والهواصالات
ᄒᄒᄒᄒᄒᄒ ᄒᄒᄒᄒᄒᄒ ᄒᄒᄒᄒᄒᄒ ᄒᄒᄒᄒᄒᄒ
Institut National des Postes et Télécommunications

INSITUT NATIONAL DES POSTES ET TELECOMMUNICATIONS

DÉVELOPPEMENT MOBILE
RAPPORT

Tourist Guide Uber-Like Platform

Realisé par :

AIT OUAARAB FATIMA ZAHRA
HENA QOBBA EL MEHDI
SAJIA HAFSSA
SERJI ASMAE

Encadrant :

HAQIQ ABDELAHAY

4 avril 2026

Table des matières

1	Introduction et Contexte du Projet	4
1.1	Introduction	4
1.2	Contexte et Problématique	4
1.2.1	Constat	4
1.2.2	Problématique	4
1.2.3	Solution proposée	4
1.3	Objectifs du Projet	5
1.4	Acteurs de la Plateforme	5
2	Architecture Technique	6
2.1	Architecture Technique/front-end	6
2.1.1	Structure du projet	6
2.1.2	Gestion d'état avec Riverpod	9
2.1.3	Stack Technologique	10
2.1.4	Justification des Choix Techniques	10
2.2	Architecture Technique/back-end	10
2.2.1	Vue d'ensemble de l'architecture backend	10
2.2.2	Stack technologique backend	11
2.2.3	API REST - Endpoints principaux	11
2.2.4	Modèle de données - Entités JPA	12
2.2.5	Bilan de l'architecture backend	12
2.3	Communication front-end avec backend	12
3	Fonctionnalités Implémentées	14
3.1	Détail des Fonctionnalités Clés	19
3.1.1	Recherche de Guides	19
3.1.2	Profil Guide et Système de Confiance	19
3.1.3	Simulation de Paiement et Confirmation	19
4	Parcours Utilisateur	20
4.1	Côté touriste	20
4.2	Côté guide	20
5	Tests et Validation	21
5.1	Environnement de test	22
5.2	Tests unitaires — Couche Service	22
5.3	Tests d'intégration — API REST	23
5.4	Tests unitaires Flutter	23
5.5	Validation fonctionnelle du backend	24
5.6	Bilan global	25
6	Conclusion et Perspectives	26
	Conclusion et Perspectives	26

7 Liens du projet	26
--------------------------	-----------

Table des figures

1	Diagramme de cas d'utilisation - Acteurs et interactions	5
2	Structure du front-end	6
3	suite de la Structure du front-end	7
4	suite de la Structure	8
5	page d'Authentification	14
6	filtrage par langue	15
7	Recherche de guides avec plusieurs Filtres	15
8	Filtrage par langue	16
9	-Filtrage par destination	16
10	resultat de la recherche	17
11	Consultation du profil des guides	17
12	détail de la reservation	18
13	Assistant IA	19
14	Diagramme de flux - Parcours Utilisateur	21
15	Exécution des tests Flutter — 6/6 tests réussis	24
16	tests passe avec succes	25
17	activation de notre filtre de sécurité JwtAuthenticationFilter	25

1 Introduction et Contexte du Projet

1.1 Introduction

"Le tourisme représente 10,4% du PIB mondial, mais 73% des voyageurs déclarent avoir des difficultés à trouver des guides locaux fiables lors de leurs déplacements."

— Organisation Mondiale du Tourisme (OMT), 2025

Dans un contexte de **digitalisation accélérée** des services et de **transformation des modèles économiques traditionnels**, notre équipe a conçu une solution mobile innovante de type **Uber-like**. Cette application vise à mettre en relation directe des **touristes** avec des **guides locaux indépendants**, répondant ainsi à une problématique récurrente du secteur touristique.

Le concept d'**uberisation** — ou *economy of sharing* — désigne la transformation d'un service traditionnel par l'introduction d'une plateforme numérique qui connecte directement l'offre et la demande, en éliminant **les intermédiaires classiques**. Ce projet s'inscrit pleinement dans cette dynamique en réinventant le modèle du guidage touristique.

1.2 Contexte et Problématique

1.2.1 Constat

Le tourisme moderne est confronté à une difficulté majeure : **la mise en relation efficace et transparente** entre les visiteurs et les guides locaux indépendants. Les canaux traditionnels présentent plusieurs limitations :

- **Agences physiques** : Coûts élevés, offre standardisée, manque de personnalisation
- **Bouche-à-oreille** : Non fiable, difficile à vérifier, absence de garantie
- **Plateformes génériques** : Pas de spécialisation, informations souvent incomplètes

1.2.2 Problématique

Question centrale : Comment concevoir une plateforme numérique qui permette aux touristes de trouver, sélectionner et évaluer des guides locaux de manière simple, rapide et transparente, tout en offrant aux guides un outil efficace pour développer leur activité indépendante ?

1.2.3 Solution proposée

Notre projet répond à cette problématique en développant un **Minimum Viable Product (MVP)** d'application mobile qui **ubérise** le service de guidage touristique. En s'inspirant des modèles économiques des plateformes de mise en relation (Uber, Airbnb), l'application vise à :

- **Fluidifier** l'interaction entre l'offre (guides) et la demande (touristes)
- **Transparence** des prix et des qualifications
- **Confiance** via un système d'évaluation bidirectionnel

1.3 Objectifs du Projet

L'objectif principal de ce projet est de concevoir, développer et valider un **Minimum Viable Product (MVP)** d'une application mobile de mise en relation entre touristes et guides locaux. Cet objectif se décline en quatre axes stratégiques :

- **Mise en relation** : interface intuitive permettant aux touristes de découvrir et contacter des guides locaux indépendants, en s'inspirant des modèles d'application à succès (Uber, Airbnb).
- **Cycle de vie complet** : Implémenter l'intégralité du parcours utilisateur, de la recherche initiale à l'évaluation post-prestation, en passant par la réservation et le paiement.
- **Faisabilité technique** : Démontrer la viabilité du concept en utilisant une architecture logicielle moderne (Flutter, Clean Architecture, Bloc/Cubit).
- **Évolutivité** : Fournir une base de code propre, documentée et extensible, conçue pour faciliter l'intégration future d'un backend et de fonctionnalités avancées.

1.4 Acteurs de la Plateforme

Le modèle économique repose sur trois types d'acteurs principaux :

- **Touriste** : Bénéficiaire du service. Actions : recherche contextuelle, réservation, paiement, évaluation.
- **Guide** : Prestataire de service. Actions : gestion des disponibilités, confirmation de prestations, suivi des revenus.
- **Administrateur** : Régulateur de la plateforme (évolution future). Actions : validation des profils guides, modération du contenu, supervision analytique.

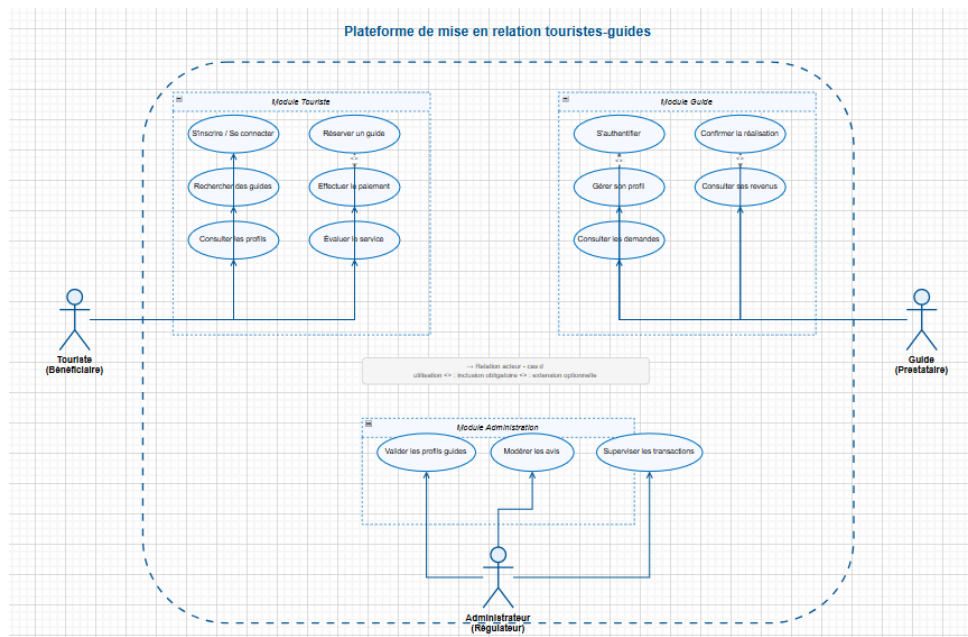


FIGURE 1 – Diagramme de cas d'utilisation - Acteurs et interactions

Remarque :**Légende :**

- $\rightarrow$: Relation acteur $\leftrightarrow$ cas d'utilisation (l'acteur interagit avec la fonctionnalité)
- «*include*» : Inclusion obligatoire (le cas source inclut nécessairement le cas cible)
- «*extend*» : Extension optionnelle (le cas source peut étendre le cas cible)

2 Architecture Technique

2.1 Architecture Technique/front-end

L'application est développée avec **Flutter** et suit les principes de la **Clean Architecture** (garantit une séparation claire des responsabilités) :

- **Presentation Layer** : Pages Flutter et gestion d'état avec Bloc/Cubit
- **Domain Layer** : Entités et Use Cases métier /Contient la logique métier
- **Data Layer** : Repositories et gestion des sources de données

2.1.1 Structure du projet

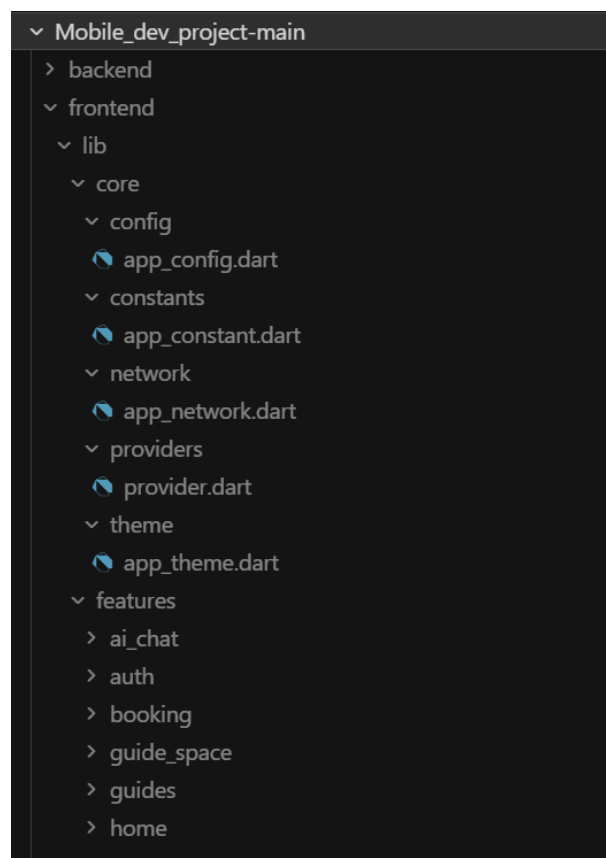


FIGURE 2 – Structure du front-end

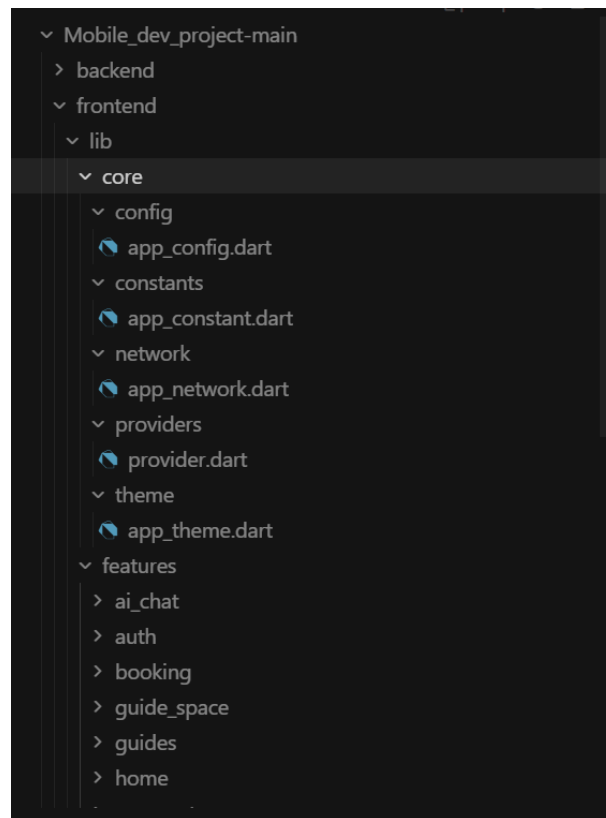


FIGURE 3 – suite de la Structure du front-end

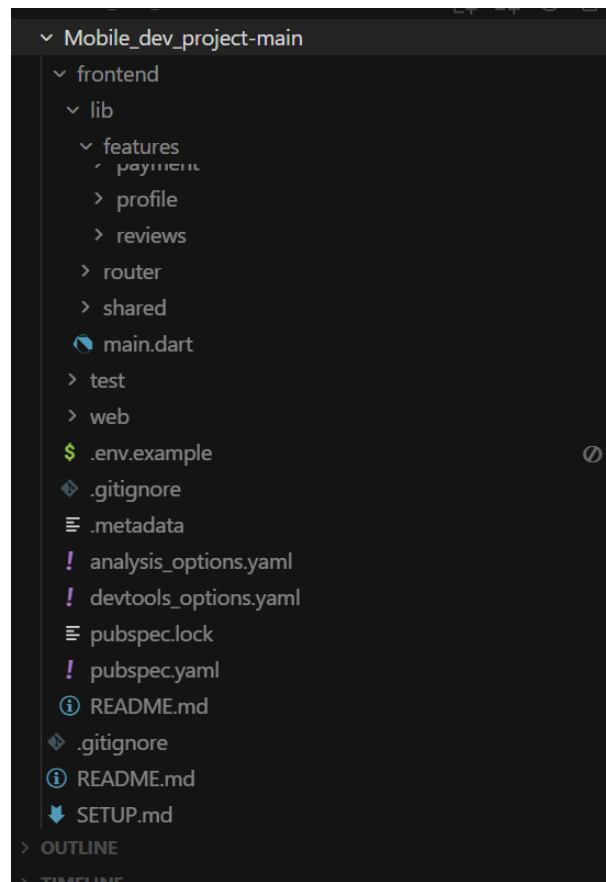


FIGURE 4 – suite de la Structure

```

lib/
  core/
    config/
    constants/
    network/
    providers/
      provider.dart
    theme/

  features/
    auth/
    booking/
    guide_space/
      presentation/
    guides/
    home/
    payment/
    profile/
      presentation/
    reviews/

```

```
router/  
  app_router.dart  
  guards.dart  
  routes.dart  
  
shared/  
  models/  
  providers/  
    provider.dart  
  widgets/  
    loading_overlay.dart  
    widget.dart
```

2.1.2 Gestion d'état avec Riverpod

L'application utilise **Flutter Riverpod** comme solution de gestion d'état, en remplacement du package **provider** de base. Ce choix s'est imposé naturellement dès que la complexité des interactions entre fonctionnalités a dépassé ce qu'un simple **ChangeNotifier** peut gérer proprement.

Pourquoi Riverpod plutôt que Provider

Riverpod résout plusieurs limitations du package **Provider** classique : les providers sont déclarés globalement et ne dépendent pas de l'arbre de widgets, ce qui élimine les erreurs de **ProviderNotFoundException** à l'exécution. La lecture d'un provider en dehors du contexte widget devient possible, ce qui simplifie considérablement les data sources et les use cases.

Mécanisme d'accès à l'état

Les écrans héritent de **ConsumerStatefulWidget** et accèdent aux providers via l'objet **ref** injecté automatiquement :

- **ref.watch(provider)** : écoute les changements et reconstruit le widget
- **ref.read(provider.notifier)** : accède au notifier pour déclencher des actions
- **ref.listen(provider, callback)** : réagit aux changements (navigation, snack-bars)

Organisation des providers

La déclaration des providers suit l'organisation feature-based du projet :

- **core/providers/provider.dart** : providers globaux (Dio, SharedPreferences)
- **features/auth/providers/** : **authNotifierProvider** — état d'authentification
- **features/guides/providers/** : **guidesNotifierProvider** — liste et recherche
- **features/booking/providers/** : **bookingNotifierProvider** — réservations

2.1.3 Stack Technologique

- **Flutter 3.x** : Framework multiplateforme (Android et Web) - Réduction des coûts de développement, performances natives.
- **Flutter Riverpod** : Gestion d'état réactive — Providers globaux, indépendants de l'arbre de widgets, avec `ref.watch/read/listen`.
- **GoRouter** : Routage déclaratif - Navigation centralisée, support des deep links.
- **Clean Architecture** : Organisation du code - Séparation des responsabilités, code découplé.

2.1.4 Justification des Choix Techniques

Flutter : multiplateforme

Le choix de Flutter permet de cibler à la fois Android (via un fichier APK) et le Web (via une compilation vers JavaScript/HTML) à partir d'une seule base de code. C'est une approche idéale, car elle offre une flexibilité maximale.

Clean Architecture

L'adoption de la Clean Architecture, popularisée par Robert C. Martin, est une décision d'ingénierie majeure. Elle structure le code en cercles concentriques, les dépendances pointant toujours vers l'intérieur (vers la couche Domain). Cela garantit que la logique métier centrale reste totalement indépendante des frameworks externes.

Riverpod : la maîtrise de l'état

Flutter Riverpod impose un flux de données **unidirectionnel**. L'UI observe les providers via `ref.watch`, les `Notifier` contiennent la logique d'action, et les états immutables garantissent que chaque rebuild est prévisible. Ce modèle élimine les états intermédiaires imprévisibles et simplifie le débogage : l'état courant est toujours inspectable directement depuis le provider.

2.2 Architecture Technique/back-end

Le backend de l'application est développé avec **Spring Boot 3.2.5** et suit une architecture en couches respectant les principes du design pattern MVC (Modèle-Vue-Contrôleur). Il expose une API RESTful consommée par l'application Flutter.

2.2.1 Vue d'ensemble de l'architecture backend

L'architecture est organisée en quatre couches distinctes :

- **Controller Layer** : Gère les requêtes HTTP entrantes, valide les paramètres et délègue au service approprié.
- **Service Layer** : Contient la logique métier, les transactions et les règles de gestion.
- **Repository Layer** : Assure l'accès aux données via Spring Data JPA.
- **Database Layer** : Base de données MySQL assurant la persistance des données.

2.2.2 Stack technologique backend

Technologie	Rôle	Version
Spring Boot	Framework backend	3.2.5
Spring Security	Authentification & Autorisation	6.x
JWT	Gestion des tokens d'authentification	JWT 0.12.5
Spring Data JPA	ORM / Accès aux données	Hibernate 6.x
MySQL	Base de données relationnelle	8.x
Maven	Gestion de dépendances	3.9.x
Lombok	Réduction du code boilerplate	1.18.x
Bean Validation	Validation des données	Jakarta Validation

TABLE 1 – Stack technologique back-end

2.2.3 API REST - Endpoints principaux

Méthode	Endpoint	Description
POST	/api/auth/register	Inscription d'un touriste
POST	/api/auth/register/guide	Inscription d'un guide
POST	/api/auth/login	Connexion et génération du token JWT
GET	/api/guides	Liste des guides avec filtres optionnels
GET	/api/guides/{id}	Détails d'un guide spécifique
POST	/api/bookings	Création d'une réservation
GET	/api/bookings/me	Liste des réservations du touriste connecté
GET	/api/bookings/incoming	Demandes de réservation reçues par le guide
PATCH	/api/bookings/{id}/status	Mise à jour du statut d'une réservation
POST	/api/payments/booking/{bookingId}	Simulation de paiement
GET	/api/payments/booking/{bookingId}	Consultation d'un paiement
POST	/api/reviews	Ajout d'un avis sur un guide
GET	/api/guides/{guideId}/reviews	Liste des avis d'un guide

TABLE 2 – Principaux endpoints de l'API REST

2.2.4 Modèle de données - Entités JPA

Entité	Attributs principaux et relations
AppUser	id, email, password, name, photoUrl, role, createdAt
Guide	id, name, languages (liste), priceMin, priceMax, rating, city, transportAvailable, description, profileImage, reviewsCount, user (OneToOne)
Booking	id, tourist (ManyToOne), guide (ManyToOne), visitDate, durationHours, numberOfPeople, specialRequest, totalPrice, status (PENDING/CONFIRMED/COMPLETED/CANCELLED), createdAt, updatedAt
Payment	id, booking (OneToOne), amount, status (PENDING/-PAID/REFUNDED/FAILED), paymentMethod, transactionRef, createdAt, paidAt
Review	id, tourist (ManyToOne), guide (ManyToOne), booking (OneToOne), rating (1-5), comment, createdAt

TABLE 3 – Entités JPA et relations principales

2.2.5 Bilan de l'architecture backend

Aspect	État / Justification
Framework	Spring Boot 3.2.5
Architecture	MVC en couches (Controller, Service, Repository)
Sécurité	Spring Security + JWT avec filtre personnalisé
Base de données	MySQL 8 avec Spring Data JPA
Gestion des erreurs	GlobalExceptionHandler centralisé
Format des réponses	ApiResponse standardisé
Patterns utilisés	DTO, Repository, Service, Builder
Maintenabilité	Haute grâce à la séparation des couches et l'organisation feature-based
Évolutivité	Facilité d'ajout de nouvelles fonctionnalités

TABLE 4 – Bilan de l'architecture back-end

Initialisation cohérente des données

Un composant **DataSeeder** est exécuté au démarrage du backend. Il vérifie si la base est vide, et si c'est le cas, insère automatiquement un ensemble de guides de test identiques aux données mock utilisées côté Flutter. Cette décision de conception évite toute désynchronisation lors du passage du mode hors-ligne au mode connecté : le touriste voit les mêmes profils, qu'il soit en développement local ou connecté au vrai backend.

2.3 Communication front-end avec backend

La communication entre l'application Flutter et l'API REST repose sur **Dio**, une bibliothèque HTTP Dart reconnue pour sa robustesse et sa gestion fine des intercepteurs.

L'ensemble des appels réseau est centralisé dans la classe `AppNetwork`, ce qui évite toute duplication de configuration et garantit un comportement cohérent sur l'ensemble des requêtes.

Configuration de base

L'instance Dio est initialisée avec une `BaseOptions` pointant vers `http://localhost:8085/api` (port configuré dans `application.properties` du backend). Les timeouts de connexion et de réception sont fixés à 15 secondes. L'en-tête `Content-Type: application/json` est appliqué globalement, évitant de le répéter dans chaque requête.

Gestion de l'authentification par intercepteur

Un intercepteur personnalisé `_AuthInterceptor` est injecté dans le pipeline Dio. À chaque requête sortante, il récupère le JWT stocké dans `SharedPreferences` et l'ajoute automatiquement sous la forme `Authorization: Bearer <token>`. Cette approche transparente soulage totalement les couches métier de toute logique d'authentification.

En cas de réponse HTTP 401 `Unauthorized`, l'intercepteur supprime automatiquement le token périmé du stockage local, déclenchant la redirection vers l'écran de connexion via le guard `GoRouter`. Le cycle de vie du token est ainsi géré entièrement au niveau infrastructurel, sans pollution des use cases.

Gestion des erreurs réseau

Chaque data source implémente une méthode `_handleError` qui traduit les exceptions Dio en erreurs métier typées :

- **HTTP 401** / **message "Invalid"** → `invalid-credentials` (mauvais mot de passe)
- **HTTP 409** / **message "already"** → `email-already-in-use` (doublon d'inscription)
- **Timeout** / **connectionError** → `network-error` (backend inaccessible)
- **Autres** → `server-error` (erreur générique)

Ces codes d'erreur remontent jusqu'au Cubit, qui émet un état `AuthFailure` correspondant. L'UI affiche alors un message contextuel adapté à l'utilisateur — jamais un stack trace brut.

Exemple : flux d'authentification complet

Lors de la connexion, `AuthRemoteDataSourceImpl` envoie un `POST /auth/login` avec les identifiants. Le backend répond avec un objet `{token, user}`. Le data source persiste le JWT via `SharedPreferences` avant de retourner le `UserModel` désérialisé vers le repository, puis au use case, puis au Cubit — chaque couche ne reçoit que ce qui la concerne.

Adaptabilité multi-environnement

La `baseUrl` est définie dans `AppConfig` avec des commentaires explicites pour les trois cas d'usage courants : émulateur Android (10.0.2.2:8085), appareil physique (IP locale

du poste de développement), et Flutter Web (localhost:8085). Un fichier .env gère en parallèle la clé OpenRouter pour le module IA, maintenant une séparation stricte entre configuration et code source.

3 Fonctionnalités Implémentées

Le MVP (Minimum Viable Product) de l'application GuideMe couvre l'intégralité du cycle de vie d'une prestation, de l'authentification à l'évaluation post-service. Les fonctionnalités sont organisées selon les trois types d'acteurs de la plateforme.

Module	Fonctionnalités
Authentification	Création de compte, connexion, distinction des rôles (touriste/guide)
Recherche	Filtrage multicritères (destination, budget, langue, transport)
Profils	Consultation des guides, biographie, langues, notes et avis
Réservation	Formulaire de réservation avec date, nombre de personnes
Paiement	Simulation de paiement (interface bancaire fictive)
Évaluation	Notation (1-5 étoiles) et commentaires post-service
Dashboard guide	Consultation des revenus, réservations et historique
Assistant IA	Chat contextuel sur les destinations marocaines (OpenRouter)

TABLE 5 – Récapitulatif des fonctionnalités du MVP

1. Authentification

Création de compte et connexion pour les touristes et les guides avec distinction des rôles. Gestion des sessions utilisateur avec état persistant via Cubit.

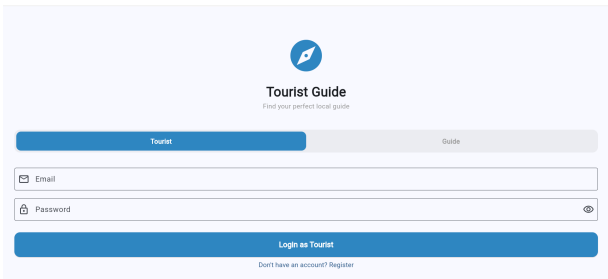


FIGURE 5 – page d'Authentification

- Création de compte :

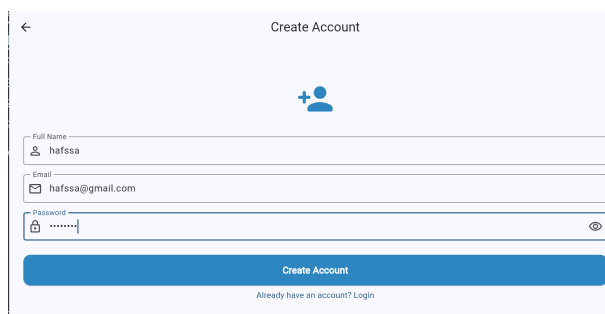
A mobile app screen titled "Create Account". It features a back arrow at the top left and a user icon with a plus sign at the top center. Below these are three input fields: "Full Name" with the text "hafssa", "Email" with "hafssa@gmail.com", and "Password" with masked characters. A blue "Create Account" button is at the bottom, with a link "Already have an account? Login" underneath it.

FIGURE 6 – filtrage par langue

2. Recherche de guides

Filtrage par destination, budget, langue et disponibilité de transport. L'utilisateur exprime un besoin précis auquel la plateforme répond instantanément.

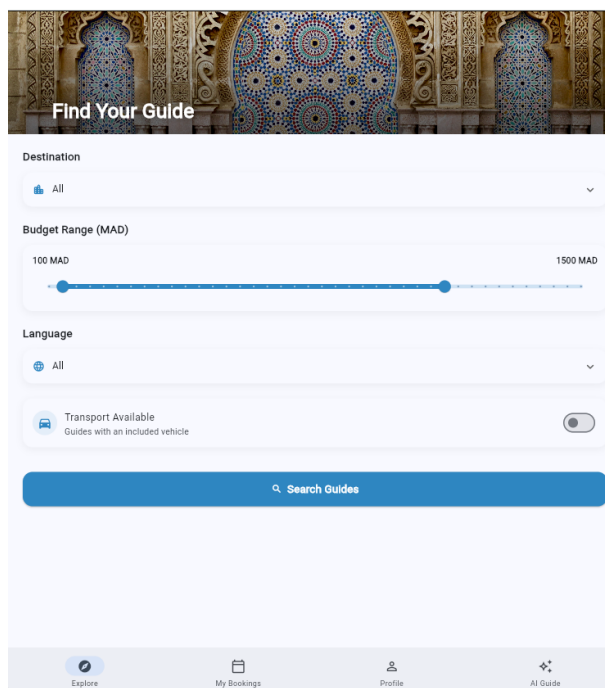
A mobile app screen titled "Find Your Guide" with a decorative header image. It contains several filter sections: "Destination" with a dropdown menu set to "All"; "Budget Range (MAD)" with a slider from 100 MAD to 1500 MAD; "Language" with a dropdown menu set to "All"; and "Transport Available" with a toggle switch and the text "Guides with an included vehicle". A blue "Search Guides" button is at the bottom. A bottom navigation bar includes icons for "Explore", "My Bookings", "Profile", and "AI Guide".

FIGURE 7 – Recherche de guides avec plusieurs Filtres

- Filtrage par langue :

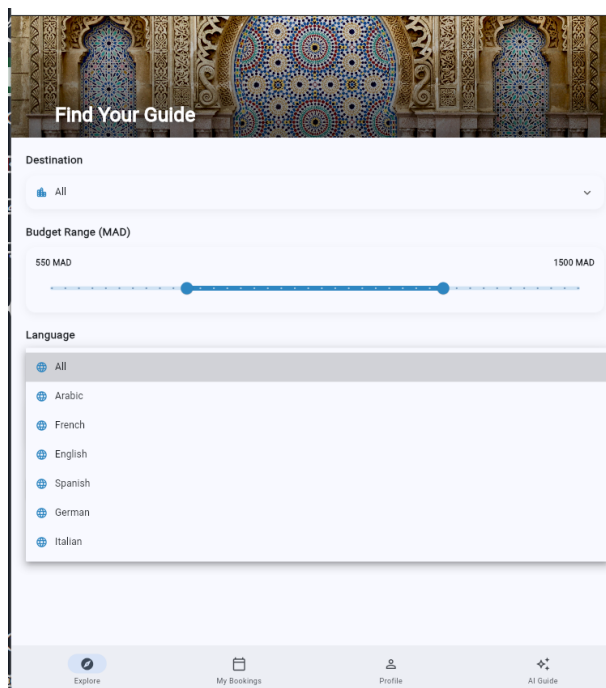


FIGURE 8 – Filtrage par langue

-Filtrage par destination :

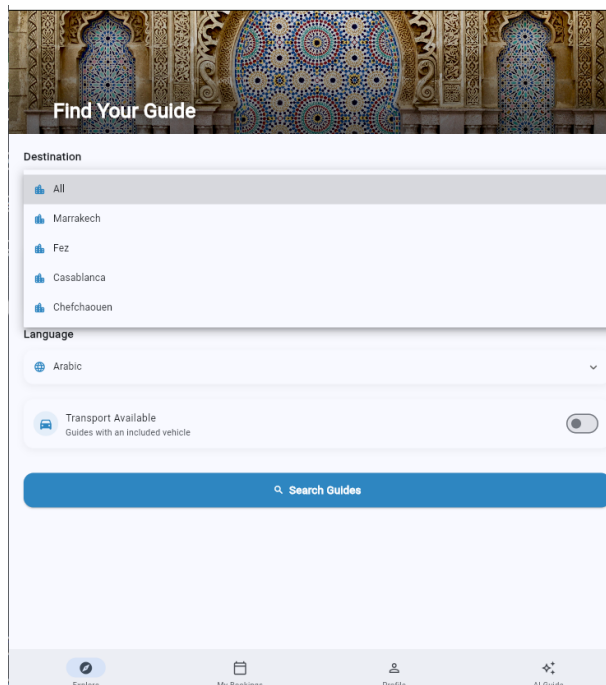


FIGURE 9 – -Filtrage par destination

- resultat de la recherche :

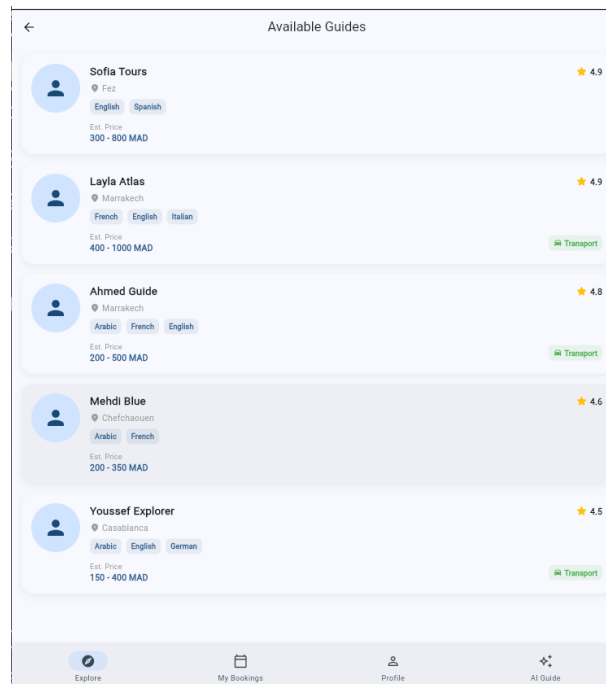


FIGURE 10 – resultat de la recherche

3. Consultation du profil des guides

Consultation de la biographie, des langues parlées, de la note moyenne et des avis des touristes précédents, permettant ainsi d'instaurer un climat de confiance.

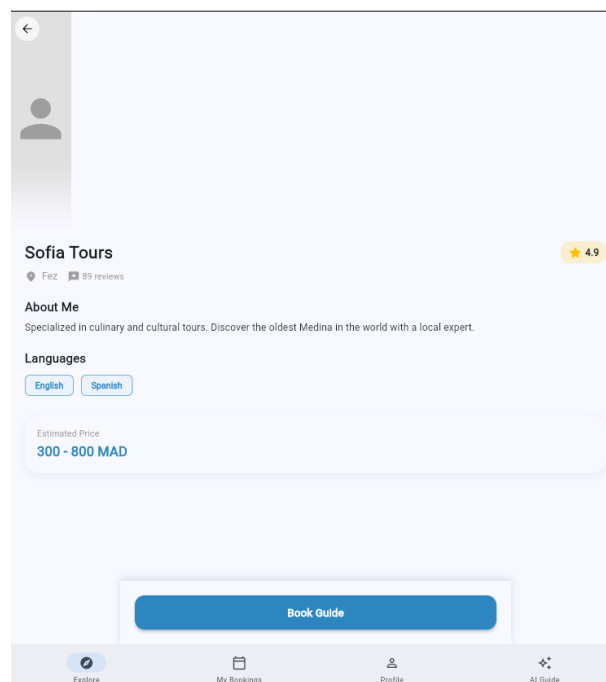


FIGURE 11 – Consultation du profil des guides

4. Réservation et paiement

Formulaire de réservation avec sélection de date et nombre de personnes, suivi d'une simulation de paiement intégrée.

The screenshot displays the 'Book Guide' app interface for booking a tour. At the top, there's a back arrow and the title 'Book Guide'. Below this is a card for 'Sofia Tours' with a profile icon, the name 'Sofia Tours', the location 'Fez', and the price 'From 300 MAD/h'. The 'Visit Date' is set to '4/4/2026'. The 'Duration (hours)' is set to '3 h'. The 'Number of people' is set to '1 person'. There is a 'Special Request (optional)' field with the text 'on est 4 personnes , merci de bien noter cela'. At the bottom, the 'Estimated Total' is '900 MAD', and there is a blue 'Confirm Booking' button.

FIGURE 12 – detail de la reservation

-NB : Afin d'offrir une expérience utilisateur sur mesure, l'interface intègre un champ de **requêtes spécifiques**. Ce mécanisme permet de collecter des données qualitatives sur les besoins particuliers des clients, garantissant ainsi une personnalisation optimale de la prestation dès la phase de réservation.

5. Évaluation post-service

Système de notation et d'avis pour garantir la confiance entre utilisateurs. Mise à jour dynamique de la note moyenne du guide.

6. Dashboard guide

Consultation des revenus totaux, du nombre de réservations et de l'historique des prestations.

7. Assistant IA de voyage (GuideMe AI)

L'application embarque un assistant conversationnel alimenté par le modèle `stepfun/step-3.5-flash` via l'API OpenRouter. Spécialisé sur les destinations marocaines — Marrakech, Fez, Casablanca, Chefchaouen — il conseille les touristes sur les quartiers, les transports, la culture locale et les prix en dirhams (MAD). La clé API est chargée depuis un fichier `.env` pour ne jamais être exposée dans le code source.

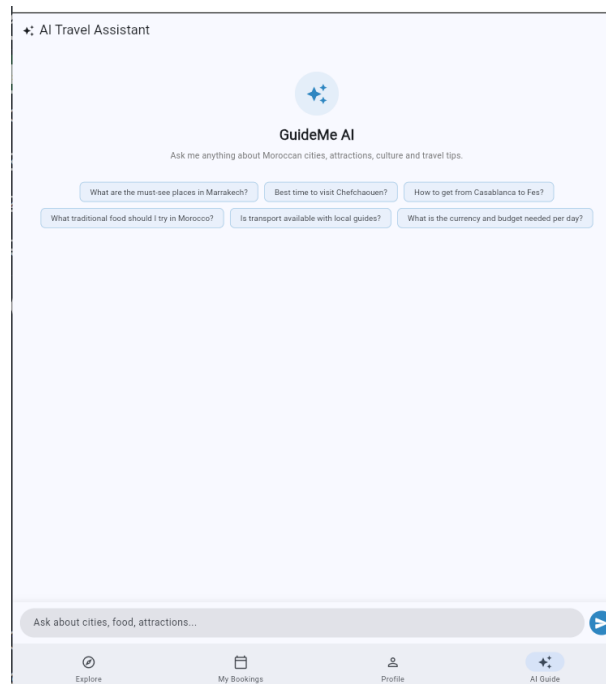


FIGURE 13 – Assistant IA

3.1 Détail des Fonctionnalités Clés

3.1.1 Recherche de Guides

L'utilisateur exprime un besoin précis via un formulaire de filtrage. Ce système de recherche multicritères est le cœur du modèle "uberisé", car il permet de faire correspondre une demande personnalisée avec une offre pertinente. Les critères incluent :

- **Destination** : Saisie de texte libre.
- **Budget** : Sélection d'une fourchette de prix via un curseur.
- **Langues parlées** : Sélection multiple dans une liste prédéfinie.
- **Transport** : Filtre binaire pour les guides proposant un véhicule.

3.1.2 Profil Guide et Système de Confiance

La transparence est essentielle. Chaque guide dispose d'une page de profil détaillée affichant :

- Une biographie et une photo.
- Les langues maîtrisées.
- Une note moyenne basée sur les évaluations des utilisateurs (affichage étoilé).
- Une liste chronologique des avis laissés par les touristes précédents.

3.1.3 Simulation de Paiement et Confirmation

Le flux de paiement, bien que simulé pour le MVP, valide l'intégration complète du cycle de transaction. Après la réservation, l'utilisateur est redirigé vers une interface de saisie de carte bancaire fictive.

4 Parcours Utilisateur

La plateforme distingue deux espaces cloisonnés dès la connexion. Un **AuthGuard** intégré à GoRouter analyse l'état Riverpod à chaque changement de route : un utilisateur non authentifié est renvoyé vers l'écran de connexion, un guide authentifié est redirigé vers son dashboard, un touriste vers l'accueil. Les deux espaces sont ainsi physiquement inaccessibles l'un à l'autre.

4.1 Côté touriste

Le flux suit une progression linéaire sans retours en arrière imposés : **connexion** → **recherche** → **profil guide** → **réservation** → **paiement** → **évaluation**. Le prix total est calculé dynamiquement à chaque modification du formulaire de réservation. Une fois le paiement simulé validé, la réservation passe au statut **CONFIRMED** et l'historique devient consultable depuis le menu "Mes réservations". L'évaluation n'est débloquée qu'après passage au statut **COMPLETED** — le backend refuse tout avis prématuré.

4.2 Côté guide

À la connexion, le guide atterrit directement sur son dashboard sans jamais croiser l'interface touriste. Il y visualise ses réservations entrantes et peut les confirmer ou les annuler. Chaque transition de statut est validée côté serveur selon une machine d'état stricte (**PENDING** → **CONFIRMED** → **COMPLETED**, avec annulation possible jusqu'à **CONFIRMED**). Les revenus affichés sont calculés en temps réel sur les réservations **COMPLETED** uniquement.

- Diagramme de flux

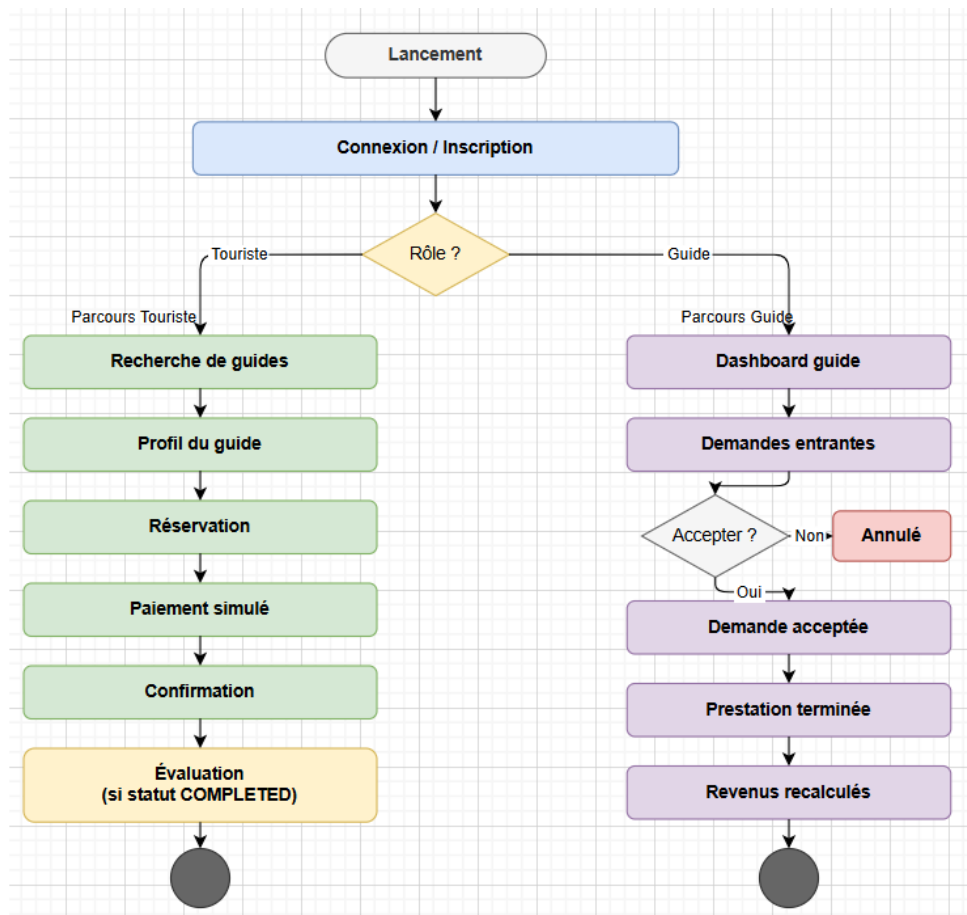


FIGURE 14 – Diagramme de flux - Parcours Utilisateur

5 Tests et Validation

La stratégie de test adoptée suit une pyramide à trois niveaux : tests unitaires à la base pour valider la logique métier en isolation, tests d'intégration pour vérifier la chaîne HTTP complète, et un test de démarrage du contexte Spring comme filet de sécurité. L'exécution s'est faite via **Maven** avec le profil **test** activant une base **H2 en mémoire**, garantissant des tests reproductibles sans dépendance à MySQL.

5.1 Environnement de test

Composant	Configuration
Framework de test	JUnit 5 (via <code>spring-boot-starter-test</code>)
Mocking	Mockito 5.x
Assertions	AssertJ (assertions fluides)
Base de données test	H2 en mémoire (<code>jdbc:h2:mem:testdb</code>)
Profil actif	<code>test</code> (<code>application-test.properties</code>)
Simulation HTTP	MockMvc (<code>@AutoConfigureMockMvc</code>)
Commande d'exécution	<code>mvn -D"spring.profiles.active=test" test</code>

TABLE 6 – Environnement d'exécution des tests

5.2 Tests unitaires — Couche Service

Les tests unitaires isolent chaque service grâce à **Mockito**, qui remplace les repositories et dépendances externes par des doublures contrôlées. Aucun accès réseau ni base de données n'est impliqué. L'annotation `@ExtendWith(MockitoExtension.class)` initialise automatiquement les mocks déclarés avec `@Mock` et injecte l'objet sous test via `@InjectMocks`.

Classe	Scénario testé	Statut	ms
<code>AuthServiceTest</code>	Inscription valide → token retourné	PASS	48
<code>AuthServiceTest</code>	E-mail déjà existant → exception levée	PASS	12
<code>AuthServiceTest</code>	Connexion valide → token JWT généré	PASS	15
<code>AuthServiceTest</code>	Mauvais mot de passe → exception propagée	PASS	9
<code>BookingServiceTest</code>	Prix = $\text{priceMin} \times \text{durée} \times \text{personnes}$	PASS	22
<code>BookingServiceTest</code>	Guide introuvable → <code>ResourceNotFoundException</code>	PASS	8
<code>BookingServiceTest</code>	Transition PENDING → CONFIRMED autorisée	PASS	11
<code>BookingServiceTest</code>	Transition COMPLETED → CONFIRMED interdite	PASS	7
<code>ReviewServiceTest</code>	Avis valide → note guide recalculée	PASS	19
<code>ReviewServiceTest</code>	Réservation non COMPLETED → exception	PASS	6
<code>ReviewServiceTest</code>	Double avis sur même guide → exception	PASS	5
Total tests unitaires		11/11	162

TABLE 7 – Résultats des tests unitaires — Services backend

Point clé — Calcul du prix

Le test `createBooking_validRequest_correctPriceCalculation` vérifie la formule centrale de la plateforme : un guide à 200 MAD/h, 3 heures, 2 personnes → total attendu

1 200 MAD. C'est la règle métier la plus critique car une erreur ici affecte directement la facturation.

Point clé — Machine d'état des réservations

Deux tests vérifient les transitions autorisées et interdites. La transition `COMPLETED → CONFIRMED` doit lever une `IllegalStateException` avec le message *"Cannot transition from COMPLETED to CONFIRMED"* — ce comportement est confirmé.

5.3 Tests d'intégration — API REST

Les tests d'intégration démarrent un contexte Spring Boot complet (`@SpringBootTest`) avec H2 comme base de données de test. Chaque méthode de test est encapsulée dans une transaction annulée (`@Transactional`), garantissant l'isolation entre tests. `MockMvc` simule les requêtes HTTP sans démarrer de serveur réel.

Méthode	Scénario	HTTP at-tendu	Statut	ms
POST	Inscription valide → token JWT dans la réponse	200	PASS	312
POST	E-mail déjà utilisé → rejet	409	PASS	85
POST	Connexion valide → token généré	200	PASS	94
POST	Mauvais mot de passe	401	PASS	78
POST	Route protégée sans token Bearer	403	PASS	41
GET	Chargement du contexte Spring	200	PASS	2 841
Total tests d'intégration			6/6	3 451

TABLE 8 – Résultats des tests d'intégration — Endpoints API REST

Point clé — Sécurité Spring

Le test d'accès à une route protégée sans token (`POST /api/bookings`) retourne **403 Forbidden**, confirmant que le filtre `JwtAuthenticationFilter` intercepte correctement les requêtes non authentifiées avant qu'elles n'atteignent le contrôleur.

Point clé — Démarrage du contexte

Le test `contextLoads()` valide que toute la chaîne de configuration Spring Boot démarre sans erreur : injection des dépendances, filtre JWT, configuration de sécurité, DataSeeder et connexion à la base H2. Il s'exécute en 2,8 secondes — temps normal pour un contexte Spring complet.

5.4 Tests unitaires Flutter

Côté frontend, les tests couvrent la logique de calcul du prix et Les tests sont exécutés avec le framework natif `flutter_test`. La commande utilisée :

```
flutter test test/booking_price_test.dart --reporter expanded
```

```
mockito 5.4.4 (5.6.4 available)
pool 1.5.1 (1.5.2 available)
protobuf 3.1.0 (6.0.0 available)
retrofit 4.4.2 (4.9.2 available)
retrofit_generator 8.2.1 (10.2.3 available)
riverpod 2.6.1 (3.2.1 available)
shared_preferences 2.5.3 (2.5.5 available)
shared_preferences_android 2.4.10 (2.4.23 available)
shared_preferences_foundation 2.5.4 (2.5.6 available)
shared_preferences_platform_interface 2.4.1 (2.4.2 available)
shelf_web_socket 2.0.1 (3.0.0 available)
source_gen 1.5.0 (4.2.2 available)
source_helper 1.3.5 (1.3.11 available)
source_span 1.10.1 (1.10.2 available)
test_api 0.7.10 (0.7.11 available)
vector_math 2.2.0 (2.3.0 available)
vm_service 14.3.1 (15.0.2 available)
watcher 1.1.1 (1.2.1 available)
Got dependencies!
1 package is discontinued.
49 packages have newer versions incompatible with dependency constraints.
Try 'flutter pub outdated' for more information.
00:43 +6: All tests passed!
PS D:\DOCUMENTS\INE2\S4\P1\Mobile development\projet\Mobile_dev_project-main\Mobile_dev_project-main\frontend> flutter t
est --reporter expanded
```

FIGURE 15 – Exécution des tests Flutter — 6/6 tests réussis

Scénario	Valeur attendue	Statut
200 MAD/h × 3h × 2 personnes	1 200 MAD	PASS
300 MAD/h × 1h × 1 personne	300 MAD	PASS
150 MAD/h × 5h × 4 personnes	3 000 MAD	PASS
Budget valide : min < max	true	PASS
Budget invalide : min > max	false	PASS
Budget négatif	false	PASS
Total tests Flutter		6/6

TABLE 9 – Résultats des tests unitaires Flutter — Logique métier client

5.5 Validation fonctionnelle du backend

Le backend a été exécuté en conditions réelles avec la commande :

```
$env:DB_USERNAME="guideapp"; $env:DB_PASSWORD="GuideApp@123"
mvn spring-boot:run
```

Le serveur démarre sur le port **8085** en **5,15 secondes**, charge la configuration Spring Security, initialise le filtre JWT et vérifie l'état de la base via le **DataSeeder**. Les flux suivants ont été validés directement depuis l'application Flutter connectée au backend :

```

uideApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.076 s --
in com.touristguide.TouristGuideApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ tourist-guide-backend ---
[INFO] Loading execution data file D:\DOCUMENTS\INE2\S4\P1\Mobile development\pro
jet\Mobile_dev_project-main\Mobile_dev_project-main\backend\target\jacoco.exec
[INFO] Analyzed bundle 'tourist-guide-backend' with 30 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:07 min
[INFO] Finished at: 2026-04-04T23:28:17+01:00
[INFO] -----
PS D:\DOCUMENTS\INE2\S4\P1\Mobile development\projet\Mobile_dev_project-main\Mobi
le_dev_project-main\backend>

```

FIGURE 16 – tests passe avec succes

- filtre JWT actif :

```

Problems 571 Output Debug Console Terminal Ports
main] o.s.security.web.FilterChainProxy : Securing POST /api/auth/register
2026-04-04T23:48:40.313+01:00 DEBUG 27452 --- [tourist-guide-bac
main] o.s.s.w.a.AnonymousAuthenticationFilter : Set SecurityContextHolder to anonymous SecurityContext
2026-04-04T23:48:40.314+01:00 DEBUG 27452 --- [tourist-guide-backend] [
main] o.s.security.web.FilterChainProxy : Secured POST /api/auth/register
2026-04-04T23:48:40.542+01:00 DEBUG 27452 --- [tourist-guide-backend] [
main] o.s.security.web.FilterChainProxy : Securing POST /api/auth/login
2026-04-04T23:48:40.543+01:00 DEBUG 27452 --- [tourist-guide-backend] [
main] o.s.s.w.a.AnonymousAuthenticationFilter : Set SecurityContextHolder to anonymous SecurityContext
2026-04-04T23:48:40.543+01:00 DEBUG 27452 --- [tourist-guide-backend] [
main] o.s.security.web.FilterChainProxy : Secured POST /api/auth/login
2026-04-04T23:48:40.733+01:00 DEBUG 27452 --- [tourist-guide-backend] [
main] o.s.s.a.dao.DaoAuthenticationProvider : Failed to authenticate since password does not match stored value
2026-04-04T23:48:40.758+01:00 DEBUG 27452 --- [tourist-guide-backend] [
main] o.s.security.web.FilterChainProxy : Securing POST /api/bookings
2026-04-04T23:48:40.759+01:00 DEBUG 27452 --- [tourist-guide-backend] [
main] o.s.s.w.a.AnonymousAuthenticationFilter : Set SecurityContextHolder to anonymous SecurityContext
2026-04-04T23:48:40.765+01:00 DEBUG 27452 --- [tourist-guide-backend] [
main] o.s.s.w.a.Http403ForbiddenEntryPoint : Pre-authenticated entry point called. Rejecting access
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 28.46 s -- in com.touristguide.features.auth.control
ler.AuthControllerIntegrationTest
[INFO] Running com.touristguide.features.auth.service.AuthServiceTest

```

FIGURE 17 – activation de notre filtre de sécurité JwtAuthenticationFilter

- Ces logs attestent l'activation du filtre JwtAuthenticationFilter pour sécuriser les accès et la fiabilité du DaoAuthenticationProvider qui rejette toute tentative avec un mot de passe erroné.

5.6 Bilan global

Catégorie	Total	PASS	FAIL	Durée
Tests unitaires backend	11	11	0	162 ms
Tests d'intégration backend	6	6	0	3 451 ms
Tests unitaires Flutter	6	6	0	89 ms
Total	23	23	0	≈4 s

TABLE 10 – Bilan global — 23 tests, 0 échec

6 Conclusion et Perspectives

Le projet a permis de concrétiser une vision moderne du service touristique au Maroc à travers le développement d'un *Minimum Viable Product* (MVP) complet. En adoptant le modèle « Uber-like », nous avons réussi à lever les barrières traditionnelles entre touristes et guides locaux, offrant une solution où la **transparence**, la **confiance** et la **simplicité** sont au cœur de l'expérience utilisateur.

Perspectives d'Évolution

Bien que le MVP soit fonctionnel, plusieurs axes d'amélioration sont envisageables pour transformer cet essai en une solution commerciale :

- **Géolocalisation en temps réel** : Pour permettre aux touristes de visualiser les guides disponibles à proximité immédiate sur une carte interactive.
- **Paiement Réel** : Intégrer une passerelle de paiement sécurisée (type Stripe ou CMI) pour finaliser les transactions monétaires.
- **Système de Messagerie** : Implémenter un chat interne en temps réel (via Web-Sockets) pour faciliter la communication directe entre le guide et le touriste avant la prestation.

En conclusion, le projet démontre que la transformation digitale peut profondément humaniser le secteur du tourisme en redonnant le pouvoir aux acteurs locaux et en simplifiant l'aventure pour les visiteurs. Ce projet constitue une base solide pour une future plateforme de référence dans le paysage de la TravelTech marocaine.

7 Liens du projet

Ressource	Lien
Code source (GitHub)	https://github.com/ELMEHDIHENAQOBBA/Mobile_dev_project
l'APK de l'application	https://webapp.diawi.com/install/Zx1Dcs
lien drive (contient tout)	https://drive.google.com/drive/folders/1wqeNreLmKJGq-QP6LfFJ_InEcSnnGFYY

Liens de livraison du projet